Class: SE **Subject: COA LAB** **Sem-III**

Experiment No 6

Course Outcome: CO3 **Blooms Level:L3**

Aim: To implement an 8086 assembly language program that checks whether a predefined string is a palindrome by applying string manipulation and comparison techniques.

Abstract:

Develop an 8086 assembly language program to determine whether a predefined string is a palindrome. The string should be reversed and stored in a separate memory buffer. The program must compare the original and reversed strings character by character using appropriate string comparison instructions. To facilitate this, both the data segment and extra segment should be initialized to the same memory block. Based on the comparison, the program should display a message indicating whether the string is a palindrome or not. All data, including the string, buffer, and messages, must be declared in the data segment, and the program should terminate using DOS interrupt services.

Sample Input and Output:

Case 1:
Input:MALAYALAM
Output:It is a palindrome

Case 2:
Input:APSIT
Output:It is not a palindrome

Theory: In the 8086 microprocessor, block transfer and block exchange operations involve moving or swapping sequences of data (arrays or strings) from one memory location to another. These operations are efficiently implemented using index registers such as SI (Source Index) and DI (Destination Index), often in combination with the DS (Data Segment) and ES (Extra Segment) registers.

String Instructions :
The string instructions facilitate operations on sequences of bytes or words. None of them take an explicit operand; instead, they all work implicitly on the source and/or destination strings. The current element (byte or word) of the source string is at DS:SI, and the current element of the destination string is at ES:DI. Each instruction works on one element and then automatically adjusts SI and/or DI; if the Direction flag is clear, then the index is incremented, otherwise it is decremented (when working with overlapping strings it is sometimes necessary to work from back to front, but usually you should leave the Direction flag clear and work on strings from front to back).

Class: SE **Subject: COA LAB** **Sem-III**

Operations that can be performed with string instructions:
• copy a string into another string
• search a string for a particular byte or word
• store characters in a string
• compare strings of characters alphanumerically

Moving (Copying) Strings Instructions:
MOVSB - copies contents of BYTE given by DS:SI into ES:DI
MOVSW - copies contents of WORD given by DS:SI into ES:DI
MOVSD - copies contents of DOUBLE WORD given by DS:SI into ES:DI

Storing Strings Instructions:
STOSB- copies contents of AL to BYTE address given by ES:DI.DI is incremented/decremented by 1.
STOSW - copies the contents of AX to the WORD address given by ES:DI.DI is incremented/decremented by 2.
STOSD - copies contents of EAX to the DOUBLE WORD address given by ES:DI.DI is incremented/decremented by 4.

Load String Instructions:
LODSB - moves the BYTE at address DS:SI into AL. SI is incremented/decremented by 1.
LODSW - moves the WORD at address DS: SI into AX. SI is incremented/decremented by 2.
LODSD - moves the DOUBLE WORD at address DS:SI into EAX. SI is incremented/decremented by 4.

Compare String Instructions:
CMPSB - compares BYTE at ES:DI with BYTE at DS:SI and sets flags.
CMPSW - compares WORD at ES:DI with WORD at DS:SI and sets flags.
CMPSD - compares DOUBLE WORD at ES:DI with WORD at DS:SI and sets flags.

In a block transfer, data from a source block is copied sequentially to a destination block. This is commonly achieved using the MOVSB instruction, which moves a byte from [DS:SI] to [ES:DI] and then automatically increments or decrements both SI and DI depending on the Direction Flag (DF). A loop counter (usually in CX) controls how many bytes are transferred.

In a block exchange, corresponding elements of two blocks are swapped instead of simply copied. This requires using temporary storage (usually a register like AL) and swapping the contents of [SI] and [DI] in a loop, again controlled by the CX register.

Using LODSB, STOSB, MOVSB, and loop instructions, along with proper segment and index register setup, allows efficient data handling. This experiment helps in understanding how to

manipulate arrays and memory blocks in assembly language.

Program:

```
.model small
.data
strr db "liril"
len equ $-strr
rstr db 20 dup(0)
msg1 db 10,13,"it is a palindrome $"
msg2 db 10,13,"it is not a palindrome $"
.code
start:
mov ax,@data
mov ds,ax
mov es,ax
mov si,offset strr
mov di,offset rstr
add di,len-1
mov cx,len
back:
mov al,[si]
mov[di],al
inc si
dec di
loop back
mov si,offset strr
mov di,offset rstr
mov cx,len
cld
repe cmpsb
jne down
lea dx,msg1
jmp down1
down:
lea dx,msg2
down1:
mov ah,09h
int 21h
mov ah,4ch
int 21h
end start
```

Output:

```
C:\>EXP6.EXE

it is a palindrome
```

Conclusion: The program successfully demonstrates palindrome checking in 8086 assembly language by reversing a predefined string and comparing it with the original. It uses segment registers, indexed addressing, and string comparison instructions like REPE CMPSB to determine equality. Based on the result, it displays an appropriate message, effectively validating the use of low-level string operations and control flow in assembly programming.

Class: SE **Subject: COA LAB** **Sem-III**

Exercise 1:
Aim: Write an assembly language program to find a character from a given string.

Abstract:
Design and implement an 8086 assembly language program that accepts a string and a single character from the user via keyboard input. The program should search for the entered character within the input string using appropriate string scanning instructions. It must display a message indicating whether the character is present in the string or not. The program should utilize DOS interrupts for input/output operations and apply string-related instructions such as SCASB and REPNZ for efficient comparison.